ALGORITHMIQUE EN BASH · EXERCICE DU MARDI 20 OCTOBRE

Le palindrome

Un palindrome, c'est un mot qui se lit pareil dans les deux sens : kayak, radar, été. Ça marche aussi avec des phrases, si on ignore les espaces et les majuscules : « Esope reste ici et se repose ».

Le but : un script qui dit, pour chaque mot ou phrase qu'on lui donne, si c'est un palindrome.

Le code de départ

Créez `palindrome.sh` et copiez ce code. Deux fonctions sont déjà écrites, la troisième est à compléter.

palindrome.sh

```
#!/bin/bash

nettoyer() { # minuscules, sans espaces
    echo "$1" | tr 'A-Z' 'a-z' | tr -d ' '
}

inverser() {
    local string=$1
    local reverse=""

    for (( i = ${#string} - 1; i >= 0 ;i-- )); do
        reverse+=${string:i:1}
    done

    echo "$reverse"
}

est_palindrome() {
    # kayak => kayak
    :   # à compléter à l'étape 3 (cette ligne ne fait rien)
}
```

Ne supprimez pas la ligne `:` tout de suite. Une fonction vide est une erreur en bash, et sans cette ligne, le script refuserait de démarrer.

Étape 1 · Comprendre `inverser`

Lisez la fonction `inverser`. La boucle part de la **dernière** lettre (indice `${#string} - 1`) et recule jusqu'à la première (indice 0). À chaque tour, elle ajoute la lettre à la fin de `reverse`.

Testez-la : ajoutez ces deux lignes **tout en bas** du script, puis lancez-le.

```
inverser "kayak"
inverser "bash"
```

Étape 2 · Comprendre `nettoyer`

`nettoyer` passe le texte en minuscules (`tr 'A-Z' 'a-z'`) puis retire les espaces (`tr -d ' '`). Ajoutez une troisième ligne de test :

```
nettoyer "Esope reste ici"
```

Avec les trois lignes de test, le script doit afficher :

Exemple d'exécution

```
$ ./palindrome.sh
kayak
hsab
esoperesteici
```

Étape 3 · Écrire `est_palindrome`

C'est le cœur de l'exercice. La fonction reçoit un mot dans `$1`. Elle doit répondre **vrai** si le mot est égal à son inverse, **faux** sinon.

Attention, elle ne doit rien afficher. Elle répond avec son **code de retour**, comme `est_pair` dans `mardi_fonctions.sh` : 0 pour vrai, 1 pour faux. C'est ce qui permettra d'écrire `if est_palindrome "kayak"; then`.

Remplacez la ligne `:` par trois étapes :

1. ranger le mot reçu dans une variable locale `mot` ;
2. calculer son inverse avec la fonction `inverser`, et le ranger dans une variable locale `inverse`. Pour récupérer ce qu'une fonction affiche, on utilise `$( )` : `local inverse=$(inverser "$mot")` ;
3. comparer `mot` et `inverse` avec `[[ ]]`.

Pour la comparaison, deux façons d'écrire la même chose :

```
# version longue
if [[ $mot == "$inverse" ]]; then
    return 0
else
    return 1
fi

# version courte : une fonction renvoie le code de sa dernière commande
[[ $mot == "$inverse" ]]
```

Remplacez les trois lignes de test du bas par celles-ci, puis lancez le script :

```
if est_palindrome "kayak"; then echo "kayak : oui"; else echo "kayak : non"; fi
if est_palindrome "bash"; then echo "bash : oui"; else echo "bash : non"; fi
```

Exemple d'exécution

```
$ ./palindrome.sh
kayak : oui
bash : non
```

Étape 4 · Le programme final

Supprimez les lignes de test. À la place, écrivez une boucle sur tous les arguments du script (`for phrase in "$@"`). Pour chaque phrase :

1. la nettoyer avec `nettoyer`, et ranger le résultat dans une variable `propre` ;
2. tester `propre` avec `est_palindrome` ;
3. afficher le résultat, en reprenant la phrase **d'origine**, pas la version nettoyée.

Une phrase avec des espaces doit être entre apostrophes sur la ligne de commande, sinon chaque mot est compté comme un argument séparé.

Exemple d'exécution

```
$ ./palindrome.sh kayak bash Radar 'Esope reste ici et se repose' Bonjour
« kayak » est un palindrome
« bash » n'est pas un palindrome
« Radar » est un palindrome
« Esope reste ici et se repose » est un palindrome
« Bonjour » n'est pas un palindrome
```

Si vous bloquez

- `syntax error near unexpected token '}'` : une de vos fonctions est vide. Remettez la ligne `:` dedans.
- Tout est « palindrome », même bash : `est_palindrome` fait un `echo` au lieu de répondre par son code de retour, ou elle se termine par une autre commande que la comparaison.
- Radar n'est pas reconnu : vous testez la phrase d'origine au lieu de `propre`.
- Les phrases sont coupées en morceaux : il manque les guillemets dans `"$@"`, ou les apostrophes autour de la phrase quand vous lancez le script.
- `bash -x palindrome.sh kayak` montre chaque ligne exécutée avec les valeurs des variables.

Pour ceux qui ont fini

Bonus 1. À la fin, affichez combien de palindromes ont été trouvés : « 3 palindromes sur 5 ».

Bonus 2. Testez « Engage le jeu que je le gagne ». Puis écrivez une deuxième version de `est_palindrome`, sans utiliser `inverser` : comparez la première lettre avec la dernière, la deuxième avec l'avant-dernière, et ainsi de suite. Dès que deux lettres sont différentes, la fonction peut répondre faux tout de suite, sans regarder la suite.